



RazorGuard

Summary, technical statement and workflow.

A short read: the problem in a paragraph, the technical statement in one sentence, the seven-stage control loop, how the recovery figure is measured, and the results.

Rs 1.23 cr

recovered per 2 days (mean of 8 seeds)

38.2%

of money the incidents put at risk

0/8

seeds where the router lost money

The problem

Payment failures are almost never global. They are narrow — one gateway, one method, one issuing bank — and a narrow failure is invisible on the chart everyone watches. A slice carrying 4% of volume can collapse from 96% to 40% and move the headline success rate by about two points, well inside normal daily variation. Nobody pages, and the money leaves quietly until a human spots a pattern in a support queue.

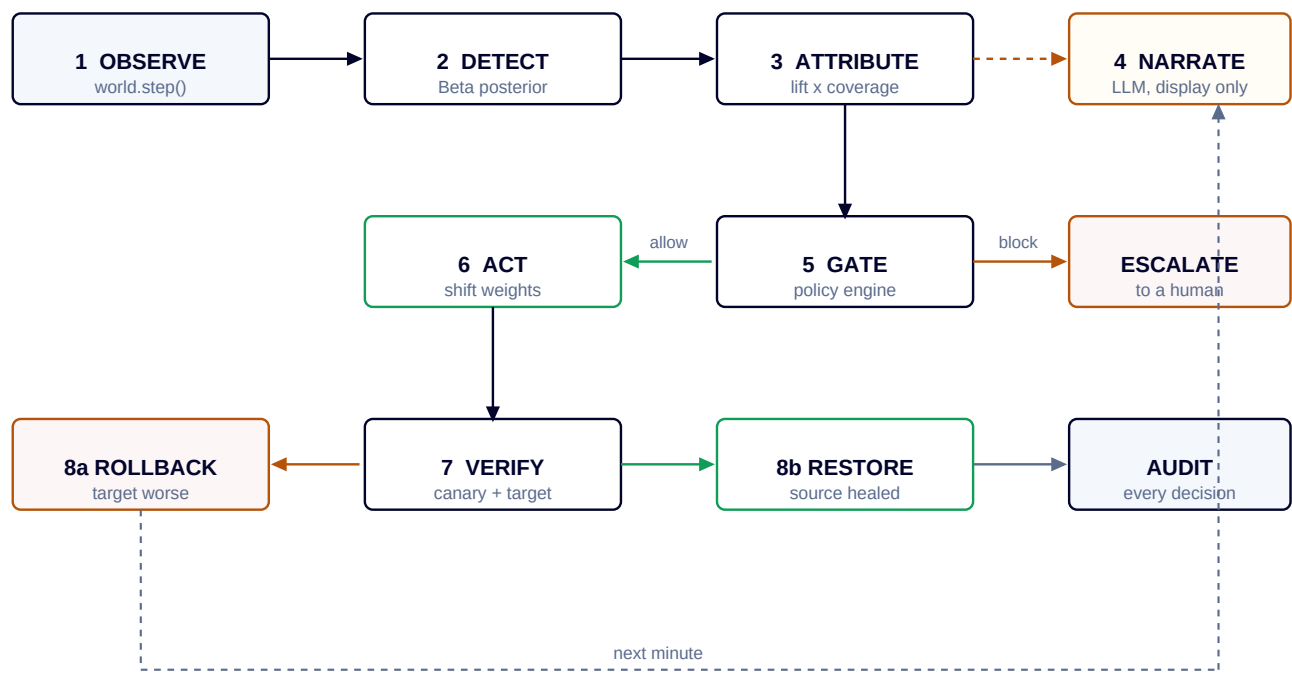
The technical statement

Detect per-slice success-rate degradation under two hostile statistical conditions — very low volume and very high test multiplicity — attribute the alarming population to a common dimension deterministically, and execute a bounded, reversible, fully audited traffic shift whose recovered revenue is measured against a control arm rather than projected.

Each clause of that sentence is a design constraint, and each was chosen against a more obvious alternative:

Constraint	Obvious approach	What is done, and why
Ground truth	Label synthetic rows, train a classifier, report precision and recall.	Outages are <i>scheduled</i> , so time-to-detect is latency against an event we caused. Training on self-generated labels measures the generator, not the world.
Detection	Threshold or z-score per slice.	Beta posterior with cohort shrinkage. A slice with four attempts a minute swings 50–100% on noise; 207,360 tests over two days drown any fixed significance level.
Attribution	Ask an LLM what is wrong.	Lift with coverage, deterministic. A sampled token in the path of a money-moving action cannot be reproduced or defended.
Action	Reroute on detection.	A separate policy engine decides <i>permission</i> . Ten bounds, stopping rules, and every refusal recorded with its rule.
Reversal	Drain the bad gateway to zero.	A 3% canary floor. A gateway with no traffic emits no observations, so you could never tell it recovered or route back.
Measurement	Count recoveries, multiply by ticket size.	Paired control and treatment on bit-identical demand, repeated across seeds, reported as a confidence interval.

The workflow

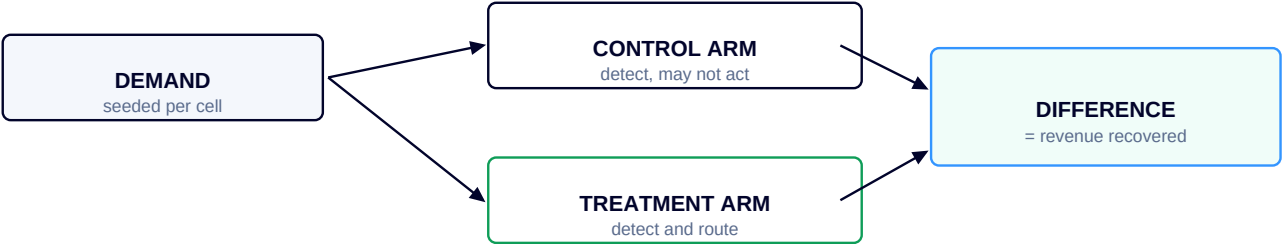


Seven stages, once a minute. The system sees only (slice, minute, attempts, successes) — never the incident plan that generated the traffic.

Stage	Guarantee it provides
1 Observe	No ground truth is visible to any decision-making code.
2 Detect	Alarms only when the drop is both statistically real and materially large.
3 Attribute	Deterministic and reproducible; refuses to name a cause the evidence cannot support.
4 Narrate	Display only. Cannot reach the ledger or any decision.
5 Gate	Ten explicit bounds; every verdict recorded with its rule, including refusals.
6 Act	Bounded fraction, per key, with a canary always left behind.
7 Verify	Rolls back if the destination proved worse; restores in thirds once the source heals.

How the measurement works

One seed, one incident plan, one demand stream



Guard: total attempts must match exactly across arms.
If they differ the run refuses to report a recovery figure at all - the comparison would be invalid.
Repeated across seeds; the headline is the confidence interval, not any single run.

Where AI is used



There are two places a model is given real latitude, and both sit on the explaining side of that line. `advisor.py` runs when the policy engine refuses and escalates: hard bounds say *no* precisely and say nothing else, so it investigates the refusal and recommends one of six courses of action for a human - contact the issuer, add capacity, manual reroute, pause automation, monitor, insufficient evidence. Choosing a destination stays arithmetic; deciding what to do when routing cannot help is judgement, and only the second is given to a model.

There is one place a model is given real latitude, and it is on the explaining side of that line. `investigator.py` is an agent that answers *why did traffic move off this gateway, and did it help?* from four read-only tools over the audit ledger and the observation stream — deciding for itself what to query and following up until it can answer. Every tool reads and none write, no decision-path module imports it, and it cannot see the incident plan either. Tests enforce all three.

Results

Rs 1.23 cr recovered per 2 days, mean of 8 paired seeds 38.2% of the money the incidents put at risk 0/8 seeds where the router lost money				
Metric	Mean	SD	Min	Max
Revenue recovered	Rs 1.23 cr	Rs 5.94 L	Rs 1.13 cr	Rs 1.31 cr
Share of exposure	38.2%	1.8%	35.6%	41.0%

Success rate gain	0.576 pp	0.019	0.542	0.604
Routing actions	99.8	4.5	92	106
Rollbacks	37.5	4.7	32	47

Net of what it cost: Rs 1.19 cr. Rerouting is not free - acquirers price differently, so incremental processing fees came to Rs 1.39 L, 1.1% of gross. UPI carries zero MDR by regulation in India, so a UPI recovery is free and a card recovery is not; the economics depend on which method degraded.

95% CI on mean recovery: **Rs 1.18 cr to Rs 1.28 cr**, which excludes zero. Cross-checked independently: measured recovery Rs 1.20 cr against an exposure reduction of Rs 1.27 cr — different quantities, agreeing within 1.5%.

What is deliberately not claimed

- **Rerouting does not always help.** A sweep across congestion curves found fleets already running at capacity, where this system loses money at every setting tested (brittle and severe). There is no spare headroom to route into, so shifting only concentrates load. The control plane now measures the realised effect of its own shifts and halts when they stop paying — which limits the damage rather than removing it. This system needs acquirers with spare capacity to be worth deploying.
- **The world is simulated, the measurement is real.** Ticket sizes, traffic mix, healthy success rates and the congestion curve are invented; DATA.md lists every one. The rupee headline scales with those ticket sizes, so a sceptical reader should take the result as +11,452 successful payments and +0.58pp of success rate, neither of which depends on a price.
- **The world is simulated.** Gateway health, demand and incidents are ours; the detector's view of them is the part under evaluation.
- **Gateway health does not degrade under load.** A real acquirer would push back as traffic arrives, so the recovery figure is an upper bound. This is the first item on the roadmap.
- **Routing weights cannot be set through Razorpay's public API.** Acquirer selection is Razorpay's own product. The executor demonstrates the half that is real — test-mode Orders carrying the decision context in their notes.
- **38% of exposure, not 90%.** Detection latency and deliberate policy caps set the ceiling.
- **About 38 rollbacks per 100 actions** are reported rather than tuned away.

Next

#	Item	Why it matters
1	Cost of action	Rerouting is not free: acquirer contracts carry volume commitments and per-transaction pricing differences. Netting that against recovered revenue turns a gross figure into a net one.
2	Varied incident plans across seeds	The current interval covers traffic randomness only. Randomising which incidents occur, and their depth and timing, would widen it honestly and make it a stronger claim.
3	A measured congestion curve	The capacity model's knee and slope are a plausible shape, not an observed one, and that curve is what sets the 80% shift cap. Fitting it to real acquirer telemetry would move the cap and is the most load-bearing unknown left.
4	Sensitivity analysis over the remaining bounds	The shift cap has now been measured and moved. The other nine are still defended by argument. The same sweep applied to each would show which earn their place.

Full detail in the companion document, *RazorGuard — Complete Project Documentation*.